

VLaRL: Augmenting Vision-Language-Action Models with Simulation-Trained Latent-Conditioned Residual RL

Namiko Saito¹, Kinam Kim^{1,2}, Heecheol Kim¹, Katsushi Ikeuchi^{1,3}, and Yasuyuki Matsushita¹

Abstract—Vision-language-action (VLA) models provide broad, instruction-conditioned manipulation behaviors, but their physical execution can remain imprecise during contact-rich interaction. Residual reinforcement learning (RL) can correct such errors while keeping the VLA frozen, but real-robot RL is costly and safety-critical. We propose VLA Latent-Conditioned RL (VLaRL), which enables residual RL for frozen VLAs to be trained in simulation and deployed on real robots without real-world RL or online adaptation. The key challenge is transferring the learned residual policy despite the visual gap between simulation and reality. Rather than requiring pixel-level visual correspondence, VLaRL uses the VLA’s internal vision-language latent representation to condition residual control and as the sim-to-real transfer interface, and learns a lightweight mapper that transforms simulation-derived latents toward the real latent distribution. Across four contact-rich manipulation tasks and two VLA backbones, VLaRL improves real-world success in all task-backbone combinations, while controlled ablations demonstrate the importance of both latent conditioning and latent alignment for transferring simulation-trained residual control. The video is available at <https://youtu.be/fCkMXTdt1gk>.

I. INTRODUCTION

Large-scale Vision-Language-Action (VLA) models have demonstrated broad, language-conditioned behaviors across diverse robot manipulation tasks [1]–[6]. Their vision-language representations enable a single policy to interpret visual observations and language instructions and generate task-relevant robot actions. However, successful real-world manipulation also requires precise physical execution, particularly during contact-rich interaction, where small errors in contact location, alignment, or sustained contact can lead to task failure. A VLA may therefore produce an appropriate nominal behavior while still failing during physical interaction. Correcting such execution errors through additional imitation learning requires collecting robot demonstration data that capture the desired corrections and updating the base policy.

This motivates us to formulate the improvement of VLA execution as a residual reinforcement learning problem: rather than relearning the complete policy, we keep the VLA frozen and learn only corrective actions for precise physical execution. Residual RL is well suited to this setting because the pretrained VLA already provides task-relevant nominal behavior, allowing the residual policy to focus on corrections that can be optimized directly from task rewards. However,

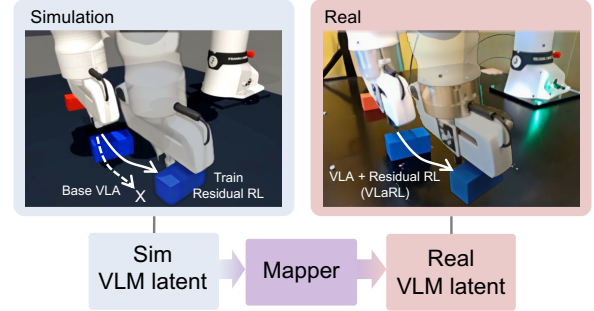


Fig. 1. Concept of VLaRL. A simulation-trained residual policy locally corrects the nominal behavior of a frozen VLA for precise physical execution. Sim-to-real alignment of the VLM latent representation enables the learned corrections to transfer to the real robot without real-world RL or online adaptation.

acquiring such corrections through RL on physical robots requires repeated interaction and introduces substantial cost and safety concerns. We therefore aim to train the residual policy entirely through simulation interaction and transfer the learned corrections to the real robot without real-world RL or online adaptation. Transferring the residual policy from simulation to reality, however, requires overcoming the visual gap between the two domains. Existing sim-to-real approaches commonly rely on either visual observations or explicit geometric states as the policy interface. Image-based policies retain rich scene information but are directly exposed to the visual sim-to-real gap [7], whereas geometric representations such as object poses provide a compact interface but require a separate perception pipeline and task-specific state specification [8]. In this work, we instead explore the internal vision-language representation already computed by the frozen VLA as the interface between simulation-trained residual control and real-world deployment. Compared with raw visual observations, this representation provides a compact feature space that abstracts visual observations while retaining task-relevant visual and language context. Reusing this representation therefore allows the residual policy to leverage the VLM’s pretrained visual abstraction rather than learning a separate visual representation from simulated images.

However, the VLM representation is not fully invariant to the visual sim-to-real gap: simulated and real observations can still produce different latent distributions. To make this representation usable as a sim-to-real interface, we learn a lightweight mapper that reduces the resulting latent distribution gap by mapping simulation-derived VLM representations toward the real distribution as shown in Fig. 1. Building on this aligned interface, we propose **VLA Latent-**

¹All authors are with Microsoft Research Asia, Tokyo, Japan. Corresponding author: Namiko Saito (namikosaito@microsoft.com).

²Kinam Kim is also with KAIST, South Korea.

³Katsushi Ikeuchi is also with The University of Tokyo, Japan.

Conditioned RL (VLaRL), which augments a frozen VLA with a latent-conditioned residual policy trained through reinforcement learning in simulation. During training, the residual policy learns corrective actions from mapped simulated latents, while at deployment it directly consumes real VLM latents and corrects the nominal VLA action. This enables residual RL to be trained entirely through simulation interaction and deployed without real-world RL or online adaptation.

We evaluate VLaRL on four contact-rich manipulation tasks—button pressing, block pushing, cup stacking, and drawer closing—using two substantially different VLA backbones, Flower [5] and GR00T N1.7 [6]. VLaRL improves real-world success in all eight task–backbone combinations without real-world RL or online adaptation. Controlled ablations further show that removing the latent mapper substantially degrades real-world performance, highlighting the importance of latent alignment for transferring simulation-trained residual control.

Our main contributions are:

- We introduce VLaRL, which augments a frozen VLA with latent-conditioned residual RL for precise physical execution, while preserving VLA’s nominal instruction-conditioned behavior.
- We use the VLM’s internal vision-language latent representation as a sim-to-real interface and learn a lightweight mapper from simulated to real latent distributions, enabling simulation-trained residual corrections to transfer to the real robot without real-world RL or online adaptation.
- We validate VLaRL across four contact-rich manipulation tasks and two substantially different VLA backbones, improving real-world success in all eight task–backbone combinations, with controlled ablations demonstrating the importance of both the VLM latent and its sim-to-real alignment.

II. RELATED WORK

A. Vision-Language-Action Models

Vision-language-action (VLA) models learn general-purpose robot policies from visual observations, language instructions, and robot demonstration data. RT-1 and RT-2 demonstrated large-scale multi-task robot learning and the transfer of pretrained vision-language representations to robotic control [1], [2], while Open X-Embodiment scaled robot learning across datasets and embodiments [9]. Recent generalist policies, including OpenVLA [4], Octo [3], Flower [5], GR00T [6], and the π -series models [10], [11], explore different architectures for mapping vision-language representations to robot actions.

VLaRL builds on these models by keeping the VLA frozen as the nominal controller and learning residual corrections through simulation-based RL.

B. Reinforcement Learning for VLA Refinement

Residual RL augments an existing controller with additive corrective actions while retaining the base policy as

a prior [12], [13], and has been applied to demonstration-based policies [14], dexterous grasping [15], and sim-to-real correction through human interventions [16]. Recent approaches combine RL with imitation or VLA policies in different ways: ResiP trains a state-based residual in simulation and distills the improved behavior into a visual policy [17], ResFiT performs residual RL on the physical robot [18], PLD uses residual RL to generate data for subsequent VLA distillation [19], and RPD optimizes an RL student initialized from VLA behavior [20]. RL Token uses an internal VLM representation as a compact interface for online RL refinement on the physical robot [11], whereas object-centric residual RL trains in simulation using estimated 6-DoF object poses as a sim-to-real interface [8].

Our VLaRL instead uses the frozen VLA’s internal vision-language representation as the residual interface while training the residual entirely in simulation, with latent mapping enabling transfer to the real robot without real-world RL.

C. Sim-to-Real Transfer and Representation Alignment

Sim-to-real robot learning addresses discrepancies in observations and dynamics [21], commonly through domain randomization [7], [22]–[25] or compact geometric representations such as 6-DoF object pose [26]. Real-to-sim approaches instead reconstruct or generate simulation environments from real observations, as in RialTo [27] and Digital Cousins [28].

VLaRL similarly uses reconstructed task configurations for simulation training but transfers residual control through the pretrained VLM representation rather than requiring pixel-level visual correspondence or an explicit object-state interface at deployment. Because pretrained representations are not necessarily invariant to the visual sim-to-real gap, VLaRL maps simulation-derived VLM latents toward the real latent distribution during residual training, allowing the deployed residual policy to directly consume real VLM latents.

III. METHOD

A. Overview

VLaRL augments a frozen VLA with a residual policy trained entirely in simulation. We treat the frozen VLA as two functional components: a vision-language module that encodes visual observations and language instructions into an internal latent representation, and an action head that generates nominal robot actions from this representation. VLaRL reuses the intermediate vision-language representation as the input to residual control while retaining the original action head as the nominal controller.

Given visual observation o_t at time t and language instruction ℓ , the vision-language module first produces a token matrix

$$Z_t = E_{\text{VLM}}(o_t, \ell), \quad Z_t \in \mathbb{R}^{S \times D}, \quad (1)$$

where E_{VLM} denotes the vision-language module within the frozen VLA, and S and D denote the number and dimension

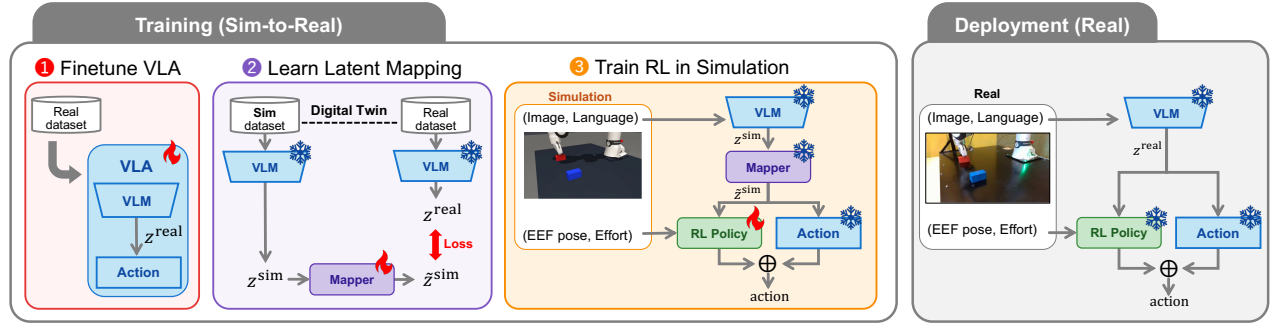


Fig. 2. Overview of VLRL and its four-stage workflow: (1) fine-tune the VLA, (2) train the latent mapper, (3) train the residual policy in simulation with the VLA and mapper frozen, and (4) deploy the frozen VLA and residual policy on the real robot. During deployment, real-image latents are supplied directly and the mapper is not used.

of tokens, respectively. Conditioned on Z_t , the VLA action head π_{act} then produces the nominal action

$$a_t^{\text{VLA}} \sim \pi_{\text{act}}(\cdot | Z_t), \quad (2)$$

which is the action generated by the frozen VLA before residual correction.

For residual control, the token matrix is reduced to a D -dimensional latent vector $z_t \in \mathbb{R}^D$ and used as the interface from the VLM to residual control. The residual policy predicts a corrective action

$$a_t^{\text{RL}} = \pi_{\text{RL}}(z_t, a_t^{\text{VLA}}, s_t, f_t), \quad (3)$$

where z_t denotes the fixed-dimensional VLM representation supplied to the residual policy, s_t is the robot proprioceptive state, and f_t is the measured Cartesian force on the wrist. The executed action is

$$a_t = a_t^{\text{VLA}} + \alpha a_t^{\text{RL}}, \quad (4)$$

where α is a fixed residual scaling factor shared across tasks.

Training this residual policy entirely in simulation introduces a visual sim-to-real gap: simulated and real observations can produce different VLM representation distributions even for similar task states. As shown in Fig. 2, VLRL addresses this gap through four steps: (1) fine-tuning and freezing the VLA and reconstructing the corresponding simulation, (2) learning a mapper that transforms simulation-derived VLM representations toward the real latent distribution, (3) training the residual policy entirely in simulation using the mapped representation, and (4) deploying the learned residual policy directly on real VLM representations without the mapper, real-world RL, or online adaptation.

B. Step 1: VLA Fine-Tuning and Digital Twin Reconstruction

For each task, we collect real-robot demonstration data and fine-tune the VLA. The fine-tuned VLA is then frozen throughout the remaining stages. Using the same demonstrations, we reconstruct approximately corresponding task configurations in simulation as digital twins, which provide the sim-real observations used for latent mapping and the simulation environment for residual RL.

C. Step 2: Sim-to-Real VLM Latent Mapping

Simulated and real observations differ visually even for similar task states, producing a gap in their VLM representations. Because the VLA is fine-tuned on real demonstrations, we learn a mapper that transforms simulation-derived representations toward the real latent distribution. The mapper serves two roles: reducing the representation shift seen by the VLA action head in simulation, and aligning the residual-policy inputs between simulation training and real-world deployment.

Let $Z_t^s, Z_t^r \in \mathbb{R}^{S \times D}$ denote the VLM token representations extracted from simulated and real observations, respectively. The mapper M_θ operates on the simulated token matrix before pooling:

$$\tilde{Z}_t^s = M_\theta(Z_t^s). \quad (5)$$

Because the reconstructed sim-real trajectories provide approximate rather than exact frame-level correspondence, we train M_θ using optimal-transport-based distribution alignment. For simulated sample i and real sample j , we define the transport cost as

$$C_{ij} = \frac{\|M_\theta(Z_i^s) - Z_j^r\|_F^2}{SD} + \lambda_\tau (\tau_i^s - \tau_j^r)^2, \quad (6)$$

where τ denotes normalized trajectory progress and λ_τ weights the trajectory-progress cost. Using this cost, we optimize the mapper with entropy-regularized optimal transport, which enables efficient computation of the transport plan using Sinkhorn iterations [29].

The mapped representation $\tilde{Z}_t^s$ is supplied to the frozen VLA action head and mean-pooled for the residual policy:

$$\tilde{z}_t^s = \text{MeanPool}(\tilde{Z}_t^s). \quad (7)$$

The mapper is frozen during residual RL.

D. Step 3: Simulation-Trained Residual Reinforcement Learning

With the mapper fixed, we train the residual policy entirely through interaction in simulation using TD3 [30]. At each step, the frozen VLA action head produces the nominal action from the mapped simulated token representation $\tilde{Z}_t^s$, while its mean-pooled representation $\tilde{z}_t^s$ is supplied to the

residual actor. The residual actor therefore receives $\tilde{z}_t^r$ together with the nominal VLA action, proprioceptive state s_t , and Cartesian force f_t , following Eq. (3).

We use an asymmetric actor-critic formulation in which the critic additionally receives privileged task state available in simulation, while the actor uses only observations available at real-world deployment. To stabilize learning around the demonstrated behavior, we additionally construct residual targets from the demonstrations:

$$a_t^{\text{RL,demo}} = \frac{a_t^{\text{demo}} - a_t^{\text{VLA}}}{\alpha}, \quad (8)$$

and use them as an auxiliary regularization term during residual-policy training. The VLA and latent mapper remain frozen throughout RL training; only the residual policy is optimized.

E. Step 4: Real-Robot Deployment

At deployment, the frozen VLA produces the nominal action a_t^{VLA} and real token representation Z_t^r from the real observation and language instruction. The real representation is directly mean-pooled as

$$z_t^r = \text{MeanPool}(Z_t^r), \quad (9)$$

and supplied to the residual policy together with the nominal VLA action, proprioceptive state s_t , and Cartesian force f_t . Because the residual policy is trained on simulated representations mapped toward the real latent distribution, the mapper is not used at deployment. All models remain frozen, and no real-world RL or online adaptation is performed.

IV. EXPERIMENTAL SETUP

A. Robot Setup and Manipulation Tasks

Experiments are conducted using a Franka Research 3 robot in both the real world and MuJoCo simulation [31]. With the real robot, we collected 32 demonstrations per task by teleoperation with a 3D mouse. To reconstruct the corresponding digital-twin configurations, we use Foundation-Pose [26] to reconstruct the object position and the environment. The same demonstrations are used for VLA finetuning, sim-real latent mapping, and demonstration-derived residual targets.

We evaluate four contact-rich manipulation tasks: button pressing, block pushing, cup stacking, and drawer closing, covering precise contact, sustained contact, spatial alignment, and articulated contact, respectively. The tasks and corresponding digital-twin observations are shown in Fig. 3, with language instructions and success criteria summarized in Table I.

We additionally evaluate unseen-object transfer for block pushing and cup stacking. The pushing policy trained on rectangular blocks is tested on novel star- and triangle-shaped objects, while the stacking policy is tested on novel sky-blue and green cups under both language-specified role assignments. No additional residual RL or real-world adaptation is performed for these objects.

B. Training Details

To evaluate the applicability of our framework across different VLA architectures, we used two VLA backbones, Flower [5] and GR00T N1.7 [6]. For Flower, we extract language-conditioned tokens from its Florence-2-based vision-language module, yielding a 1024-dimensional representation after mean pooling; for GR00T N1.7, we use its language-conditioned image tokens, yielding a 2048-dimensional representation. For both backbones, the normalized Cartesian VLA action is 7-dimensional, comprising three translations, three rotations, and one gripper command.

The latent mapper is a four-layer Transformer encoder with 16 attention heads and feed-forward dimension twice the VLA token dimension, operating on the token sequence before mean pooling. The mapper uses a learnable residual connection whose scaling factor is initialized to zero, such that it initially behaves as an identity transformation. The mapper is trained with Sinkhorn divergence using $\lambda_\tau = 0.5$, entropic regularization $\epsilon = 0.05$, and 50 Sinkhorn iterations.

The residual policy receives the normalized Cartesian VLA action, the end-effector position and roll-pitch-yaw orientation, gripper state, and 3-dimensional Cartesian force. Residual policies are trained for 60,000 environment steps using TD3 [30]. The actor and twin critics are four-layer ReLU MLPs with hidden width 1024; the actor outputs a 7-dimensional tanh residual action and is initialized to produce near-zero corrections. We use batch size 256, discount factor $\gamma = 0.99$, Polyak coefficient $\tau = 0.005$, and residual scale $\alpha = 0.10$. The actor is additionally regularized using the demonstration-derived residual targets described in Sec. III-D and the residual-action magnitude, while the critics receive privileged task state available only in simulation. The environment reward combines task progress with a terminal success bonus. The task-progress rewards encourage target contact for button pressing, sustained displacement with limited rotation for block pushing, reaching and aligning the instructed cups for cup stacking, and reducing the instructed drawer opening for drawer closing.

C. Evaluation Protocol

Initial object positions are randomized by ± 3 cm and orientations by $\pm 15^\circ$; for drawer closing, the drawer opening is randomized by ± 3 cm. Simulation results are reported over three random seeds, and each real-world condition is evaluated over 40 trials. All models remain frozen during real-world evaluation, with no real-world RL or online adaptation.

We compare the frozen VLA with VLaRL and conduct two controlled ablations using Flower. *w/o VLM latent* removes the latent while retaining the nominal VLA action, proprioception, and force, whereas *w/o Mapper* trains the residual policy on unaligned simulated VLM latents. Both ablations follow the same evaluation protocol and success criteria as VLaRL.

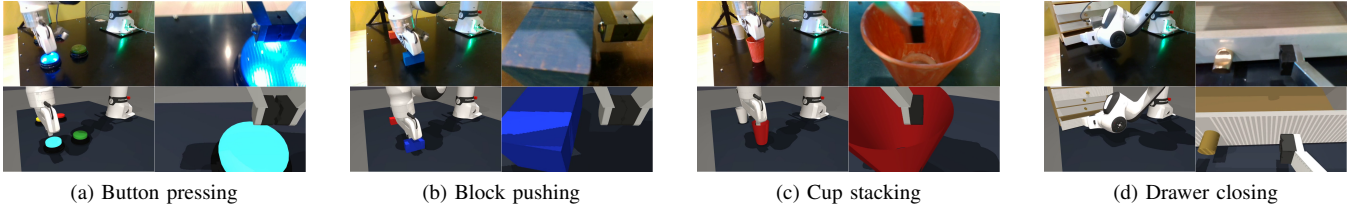


Fig. 3. The four manipulation tasks and their digital-twin observations. Within each task panel, the top row shows the real left- and hand-camera images, and the bottom row shows the corresponding simulated left- and hand-camera images extracted at the same time step.

TABLE I
EVALUATION TASKS AND SUCCESS CRITERIA.

Task	Local-control challenge	Instruction	Success criterion
Press button	Precise target contact	"Press the red / blue / yellow / green button"	Beep + light
Push block	Sustained contact	"Push the blue / red block"	Displacement > 10 cm
Stack cup	Grasp + spatial alignment	"Put the red / white cup inside the white / red cup"	Overlap > 4 cm
Close drawer	Articulated contact	"Close the upper / bottom drawer"	Opening < 1 cm

V. RESULTS

A. Simulation-Trained Residual Control Improves Real-World Execution

Figures 4 and 5 summarize the simulation and real-robot results across four tasks and two VLA backbones. In simulation, applying the mapper improves the nominal VLA success across all task-backbone combinations, consistent with reducing the representation shift encountered when the real-data-finetuned VLA is applied to simulated observations. Adding residual RL further improves success across all combinations; for example, Flower block-pushing success increases from 32.5% with the base VLA to 42.5% with the mapper and 67.5% with VLaRL.

The simulation-trained residual policies transfer to the real robot without real-world RL or online adaptation, with VLaRL improving success over the base VLA in all eight task-backbone combinations. The gains are particularly pronounced for Flower button pressing (67.5% to 100.0%), Flower block pushing (22.5% to 50.0%), and GR00T N1.7 cup stacking (17.5% to 45.0%), covering precise target contact, sustained contact, and spatial alignment, respectively. Representative Flower real-world rollouts in Fig. 6 show that the frozen VLA produces task-relevant nominal behavior but can fail during local physical execution, while our VLaRL with the residual correction enables successful completion in the illustrated examples.

Simulation and real-world performance do not always follow the same trend. For example, Flower achieves substantially higher base success on cup stacking in reality than in simulation, and the residual provides only a small additional real-world improvement. This discrepancy indicates that latent alignment does not eliminate other sim-to-real differences, such as contact dynamics and the distribution of VLA actions.

B. Ablation Analysis

We next evaluate the role of latent alignment in real-world residual transfer and the contribution of VLM-latent conditioning using Flower. As shown in Fig. 7, removing the mapper substantially degrades real-world performance.

In particular, block pushing and cup stacking fail in all 40 real-robot trials without the mapper, compared with 50.0% and 70.0% success with the full VLaRL, respectively. This result shows that residual policies trained on unaligned simulated VLM representations do not directly transfer to real representations, highlighting the importance of latent mapping for sim-to-real residual learning.

Removing the VLM latent while retaining the nominal VLA action, proprioception, and force also degrades real-world performance on all four tasks, including button pressing (100.0% to 70.0%) and block pushing (50.0% to 20.0%). This indicates that the VLM latent provides information useful for residual control beyond the nominal action and robot state.

We further analyze the residual policy using integrated gradients [32]. As shown in Fig. 9, the VLM latent receives the largest attribution to the residual-action norm across all four tasks and both VLA backbones, ranging from 72.1% to 89.7% for Flower and from 84.0% to 90.9% for GR00T N1.7. The relative contribution of the physical inputs varies across tasks: force attribution rises for block pushing, which requires sustained contact, while proprioception becomes relatively more important for cup stacking, which requires precise position adjustment. These task-dependent patterns suggest that the residual policy combines the VLM representation with physical feedback according to the local-control requirements of each task. Together with the controlled latent ablation, this analysis indicates that the learned residual policy makes substantial use of the VLM representation while also incorporating complementary physical information.

C. Unseen-Object Transfer

Although the mapper and residual policy are trained on a limited set of objects, we further evaluate VLaRL on unseen objects as shown in Fig. 8, to assess whether the residual corrections preserve the generalization of the frozen VLA. For block pushing, the unseen star- and triangle-shaped objects are evaluated across balanced target-object and left-right placement conditions. For cup stacking, the unseen sky-blue and green cups are evaluated across balanced language-specified source-target roles and left-right placements. With-

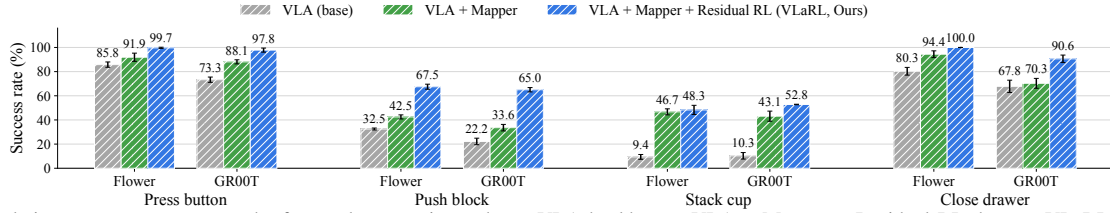


Fig. 4. Simulation success rates across the four task categories and two VLA backbones. VLA + Mapper + Residual RL denotes VLaRL (Ours). Error bars report the variation over three seeds.

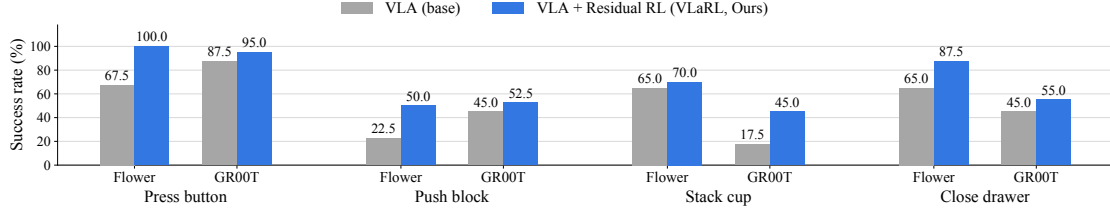


Fig. 5. Real-robot success rates over 40 trials for each task and VLA backbone. VLA + Residual RL denotes VLaRL (Ours), which improves the frozen base VLA in all eight evaluated combinations.

out additional residual RL or real-world adaptation, VLaRL improves real-world success from 40.0% (16/40) to 57.5% (23/40) for block pushing and from 55.0% (22/40) to 62.5% (25/40) for cup stacking. These results show that the learned residual corrections remain effective under unseen object variation while preserving instruction-dependent target and object-role selection from the frozen VLA.

VI. DISCUSSION

A. Why Latent-Conditioned Residual RL?

Residual control provides a way to improve precise physical execution without retraining or replacing the underlying VLA. The frozen VLA remains responsible for instruction-conditioned nominal behavior, while RL provides bounded local corrections for contact, alignment, and interaction errors. Our results across Flower and GR00T N1.7 show that this interface can be applied to different VLA designs, while the unseen-object results show that residual correction can improve execution under object variation while retaining the instruction-dependent target and object-role selection of the frozen VLA.

In VLaRL, conditioning the residual policy on the VLM latent allows it to reuse a representation derived from both visual observations and language instructions. Unlike the nominal VLA action alone, the VLM latent exposes the visual and instruction context from which the action was generated. This is particularly relevant when the same scene requires different corrections depending on the instructed target or manipulation role. The latent ablation and integrated-gradients analysis show that the residual policy makes substantial use of this representation beyond the nominal VLA action, proprioception, and force. Explicit geometric states such as object poses provide an alternative compact and interpretable interface, but require task-relevant objects and state variables to be specified and estimated in advance. VLM latents instead provide a richer interface containing appearance, semantic, and instruction context without requiring an object-pose estimator at deployment.

B. Simulation-Only Residual Learning

A central motivation of VLaRL is to avoid costly real-world RL interaction by learning residual corrections entirely in simulation. The challenge is that visual differences between simulation and reality propagate into the VLM representation, so a residual policy trained directly on simulated latents may not transfer to real latents. VLaRL addresses this problem by mapping simulation-derived representations toward the real latent distribution before residual RL training. The mapper ablation shows that this alignment is important for real-world transfer, enabling simulation-trained residual corrections to be deployed on the real robot without real-world RL or online adaptation. This provides a practical way to exploit large-scale simulation interaction for improving the physical execution of frozen VLA policies.

C. Limitations and Future Directions

Several limitations remain. First, training the latent mapper requires approximately corresponding sim–real trajectories constructed through task-specific digital-twin reconstruction. Although this process is performed only during training, reducing or eliminating the need for such paired data would make the approach easier to scale.

Second, the mapper addresses differences in VLM representations arising from the visual sim-to-real gap, but does not explicitly address discrepancies in contact dynamics, friction, compliance, or other physical properties. These remaining differences may explain discrepancies between simulation and real-world performance.

Third, the mapper is specific to each VLA backbone, and the residual RL is currently trained separately for each task category. Learning transferable latent alignment and shared residual policies across backbones and manipulation skills is an important direction toward more general residual control.

Finally, our unseen-object evaluation covers a limited range of object variations and does not establish open-world generalization. Nevertheless, the improvements on unseen objects suggest that residual learning can improve local

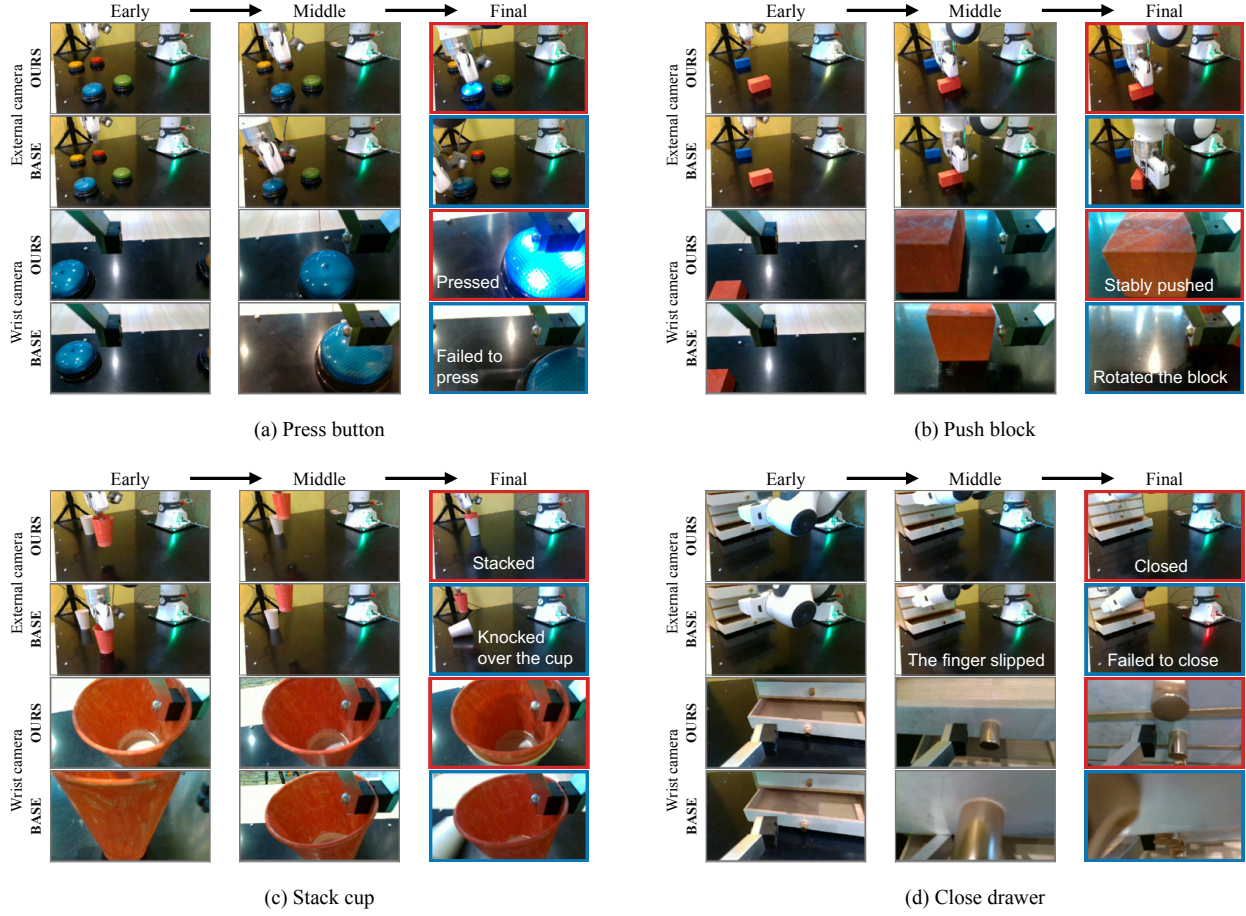


Fig. 6. Representative Flower real-robot rollout progressions for the base VLA and VLaRL across the four tasks. Columns show temporal progression from early to final states; rows show the base and proposed methods for the left and hand cameras.

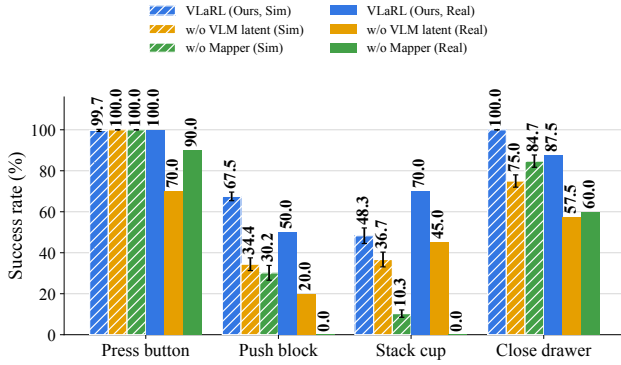


Fig. 7. Flower ablations of the VLM latent input and sim-to-real latent mapper. “VLaRL (Ours)” is the complete system; “w/o VLM latent” retains the nominal VLA action, proprioception, and force, while “w/o Mapper” uses raw simulated VLM latents. Hatched bars denote simulation and solid bars denote real-robot evaluation. Simulation error bars report variation over three seeds; real-robot results use 40 trials per task.

execution while preserving the VLA’s ability to generate instruction-dependent nominal behavior for unseen targets.

VII. CONCLUSION

We presented VLaRL, which augments a frozen VLA with latent-conditioned residual RL for precise physical execution. VLaRL reuses the VLA’s internal vision-language

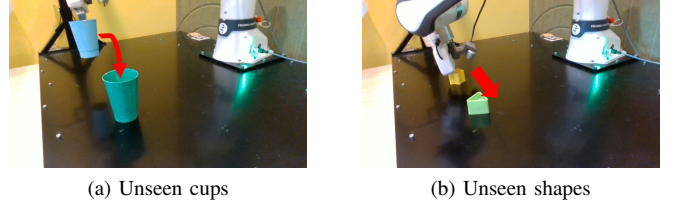


Fig. 8. Transfer to objects not used during residual-policy training. The cup experiment changes appearance and language-assigned roles, while the pushing experiment changes object geometry.

representation as the residual-control interface and learns a sim-to-real mapper that transforms simulation-derived latents toward the real latent distribution, enabling residual RL to be trained entirely in simulation and deployed without real-world RL or online adaptation. Across four contact-rich manipulation tasks and two different VLA backbones, Flower and GR00T N1.7, VLaRL consistently improved real-world success, demonstrating its applicability across different VLA architectures, while ablations demonstrated the importance of both latent conditioning and sim-to-real latent mapping. The residual policies also improved execution on unseen objects while retaining the instruction-dependent nominal behavior of the frozen VLA. These results demonstrate a practical approach to combining the generalizable behavior

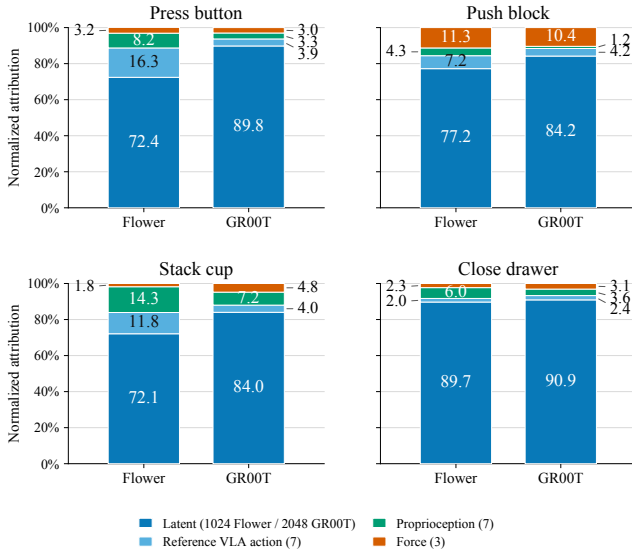


Fig. 9. Integrated-gradients attribution of the residual-action norm to the VLM latent, reference VLA action, proprioception, and force. Absolute group attributions are normalized to sum to 100%.

of pretrained VLAs with simulation-trained residual control for precise real-world manipulation.

REFERENCES

- [1] A. Brohan *et al.*, “RT-1: Robotics transformer for real-world control at scale,” *arXiv preprint arXiv:2212.06817*, 2022.
- [2] B. Zitkovich *et al.*, “RT-2: Vision-language-action models transfer web knowledge to robotic control,” in *Proceedings of the 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 229, 2023, pp. 2165–2183.
- [3] D. Ghosh *et al.*, “Octo: An open-source generalist robot policy,” in *Proceedings of Robotics: Science and Systems*, 2024.
- [4] M. J. Kim *et al.*, “OpenVLA: An open-source vision-language-action model,” in *Proceedings of the 8th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 270, 2025, pp. 2679–2713.
- [5] M. Reuss, H. Zhou, M. Rühle, O. E. Yagmurlu, F. Otto, and R. Litoutikov, “FLOWER: Democratizing generalist robot policies with efficient vision-language-flow models,” in *Proceedings of the 9th Conference on Robot Learning*, 2025.
- [6] NVIDIA *et al.*, “GR00T N1: An open foundation model for generalist humanoid robots,” *arXiv preprint arXiv:2503.14734*, 2025.
- [7] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, “Domain randomization for transferring deep neural networks from simulation to the real world,” in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2017, pp. 23–30.
- [8] K. Kim, N. Saito, H. Kim, K. Ikeuchi, J. Choo, and Y. Matsushita, “Object-centric residual rl for zero-shot sim-to-real vla enhancement,” 2026. [Online]. Available: <https://arxiv.org/abs/2606.18953>
- [9] Open X-Embodiment Collaboration *et al.*, “Open X-embodiment: Robotic learning datasets and RT-X models,” in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2024.
- [10] Physical Intelligence *et al.*, “ $\pi_{0.7}$: a steerable generalist robotic foundation model with emergent capabilities,” 2026. [Online]. Available: <https://arxiv.org/abs/2604.15483>
- [11] C. Xu, J. T. Springenberg, M. Equi, A. Amin, A. Esmail, S. Levine, and L. Ke, “RI token: Bootstrapping online rl with vision-language-action models,” 2026. [Online]. Available: <https://arxiv.org/abs/2604.23073>
- [12] T. Silver, K. Allen, J. Tenenbaum, and L. Kaelbling, “Residual policy learning,” *arXiv preprint arXiv:1812.06298*, 2018.
- [13] T. Johannink *et al.*, “Residual reinforcement learning for robot control,” in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2019, pp. 6023–6029.
- [14] M. Alakuijala, G. Dulac-Arnold, J. Mairal, J. Ponce, and C. Schmid, “Residual reinforcement learning from demonstrations,” *arXiv preprint arXiv:2106.08050*, 2021.
- [15] F. Ceola, L. Rosasco, and L. Natale, “RESPRECT: Speeding-up multi-fingered grasping with residual reinforcement learning,” *IEEE Robotics and Automation Letters*, vol. 9, no. 4, pp. 3045–3052, 2024.
- [16] Y. Jiang, C. Wang, R. Zhang, J. Wu, and L. Fei-Fei, “TRANSIC: Sim-to-real policy transfer by learning from online correction,” in *Proceedings of the 8th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 270, 2025, pp. 1691–1729.
- [17] L. Ankile, A. Simeonov, I. Shenfeld, M. Torne, and P. Agrawal, “From imitation to refinement-residual RL for precise assembly,” in *Proceedings of the 8th Conference on Robot Learning*, 2024.
- [18] L. Ankile, Z. Jiang, R. Duan, G. Shi, P. Abbeel, and A. Nagabandi, “ResFit: Residual off-policy RL for finetuning behavior cloning policies,” *arXiv preprint arXiv:2509.19301*, 2025.
- [19] W. Xiao, H. Lin, A. Peng, H. Xue, T. He, Y. Xie *et al.*, “Self-improving vision-language-action models with data generation via residual RL,” *arXiv preprint arXiv:2511.00091*, 2025.
- [20] T. Jülg, W. Burgard, and F. Walter, “Refined policy distillation: From VLA generalists to RL experts,” *arXiv preprint arXiv:2503.05833*, 2025.
- [21] W. Zhao, J. P. Queralta, and T. Westerlund, “Sim-to-real transfer in deep reinforcement learning for robotics: A survey,” *arXiv preprint arXiv:2009.13303*, 2020.
- [22] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, “Sim-to-real transfer of robotic control with dynamics randomization,” in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2018, pp. 1–8.
- [23] Y. Chebotar, A. Handa, V. Makoviychuk, M. Macklin, J. Issac, N. Ratliff, and D. Fox, “Closing the sim-to-real loop: Adapting simulation randomization with real world experience,” in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2019, pp. 8973–8979.
- [24] F. Ramos, R. Possas, and D. Fox, “BayesSim: Adaptive domain randomization via probabilistic inference for robotics simulators,” in *Proceedings of Robotics: Science and Systems*, 2019.
- [25] Y. J. Ma, W. Liang, H.-J. Wang, Y. Zhu, L. Fan, O. Bastani, and D. Jayaraman, “DrEureka: Language model guided sim-to-real transfer,” in *Proceedings of Robotics: Science and Systems*, 2024.
- [26] B. Wen *et al.*, “FoundationPose: Unified 6D pose estimation and tracking of novel objects,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.
- [27] M. Torne Villasevil, A. Simeonov, Z. Li, A. Chan, T. Chen, A. Gupta, and P. Agrawal, “Reconciling reality through simulation: A real-to-sim-to-real approach for robust manipulation,” in *Proceedings of Robotics: Science and Systems*, 2024.
- [28] T. Dai, J. Wong, Y. Jiang, C. Wang, C. Gokmen, R. Zhang, J. Wu, and L. Fei-Fei, “Automated creation of digital cousins for robust policy learning,” in *Proceedings of the 8th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 270, 2025, pp. 4912–4943.
- [29] M. Cuturi, “Sinkhorn distances: Lightspeed computation of optimal transportation,” in *Advances in Neural Information Processing Systems* 26, 2013, pp. 2292–2300.
- [30] S. Fujimoto, H. van Hoof, and D. Meger, “Addressing function approximation error in actor-critic methods,” in *Proceedings of the 35th International Conference on Machine Learning*, 2018, pp. 1587–1596.
- [31] E. Todorov, T. Erez, and Y. Tassa, “Mujoco: A physics engine for model-based control,” in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2012, pp. 5026–5033.
- [32] M. Sundararajan, A. Taly, and Q. Yan, “Axiomatic attribution for deep networks,” in *Proceedings of the International Conference on Machine Learning*, 2017, pp. 3319–3328.